MILESTONE-1

Datasets Point to Remember:

The TrustShare project does not require any external dataset because it is a secure file-sharing and document-management application, not a machine-learning project. Instead, the system works with user information, uploaded files, file metadata, and activity records generated during usage. For Milestone 1, you can create a few test accounts and upload sample files such as PDFs, Word documents, Excel sheets, and images to test the application. When a user uploads a file, the FastAPI backend validates it, stores the file in the development storage location, and saves its details, such as filename, file size, file type, upload date, and owner, in PostgreSQL. You can also use MongoDB to record activities such as logins and file uploads, while Redis can be configured for the supporting functions planned for the system. In Milestone 2, you will implement AES-256 encryption, secure key management, cloud storage integration, and secure file sharing. Therefore, you only need sample files and test-user information for Milestone 1; you do not need to download a dataset from Kaggle or any other source.

Step 1: Architecture Planning, Database Schema & Wireframes

What You Are Doing

Before writing any project logic, map out how data flows across the system, how information is structured in storage, and what the user interface looks like.

Explanation & Key Actions

- **System Architecture Mapping:** Define how the user interface (React/Next.js) communicates with the server (FastAPI) and databases (PostgreSQL, MongoDB, Redis).
- **Database Design:**
    - **Relational Data (PostgreSQL):** Plan structured tables to store accounts, passwords, email addresses, roles, and authorization rules.
    - **Document Data (MongoDB):** Plan dynamic data structures to store file information (original name, file size, owner ID, upload date).
    - **In-Memory Cache (Redis):** Set up fast temporary storage to hold active login sessions, user tokens, and rate-limiting counters.
- **UI Wireframing:** Design simple screen layouts/mockups for user login, signup, password reset, and the main workspace dashboard.

Step 2: Project Initialization & Environment Setup

What You Are Doing

Set up the workspace, folder structures, and execution environments for the backend, frontend, and database services.

Explanation & Key Actions

- **Containerization (Docker):** Create a local environment setup using Docker to run PostgreSQL, MongoDB, and Redis simultaneously without manual system installation.
- **Backend Setup:** Initialize the FastAPI application structure, configure environment variables, and establish connections to all three databases.
- **Frontend Setup:** Initialize the React/Next.js client application with Tailwind CSS for styling and Axios for handling API communications with the backend.

Step 3: Implement Authentication & Role-Based Access Control (RBAC)

What You Are Doing

Build user management and security systems so users can securely register, log in, and access feature areas based on assigned permissions.

Explanation & Key Actions

- **Account Registration & Login:** Create API pathways and logic to allow standard user registration, password hashing (so passwords are never saved in plain text), and verification.
- **Token-Based Sessions (JWT):** Implement temporary digital authorization passes (JWTs) issued upon login to keep users authenticated as they navigate pages.
- **Role-Based Access Control (RBAC):** Set permission checks to distinguish standard users, guests, and system administrators.
- **Multi-Factor Authentication (MFA) & OAuth Foundations:** Lay the groundwork for multi-step security checks and third-party login options.

Step 4: Build Secure File Upload & Storage Workflows

What You Are Doing

Build the core backend processes that accept file uploads from users, inspect them for safety, store them temporarily, and log their details.

Explanation & Key Actions

- **File Validation:** Build checks to inspect incoming file requests for authorized file types (e.g., PDFs, images, text) and file size limitations.

- **File Intake & Local Storage:** Handle the stream of incoming file data and save files to designated storage paths on the server. (Note: Encryption and cloud S3 storage are integrated in Milestone 2).
- **Metadata Recording:** Capture information about each uploaded file (file size, original name, uploading user, timestamp) and save those metadata records into MongoDB.

Step 5: Develop File Management Dashboard (Frontend UI)

What You Are Doing

Build the user-facing web interface where users can interact with the authentication system and manage uploaded files.

Explanation & Key Actions

- **Auth Screens:** Build visual screens for user login, account registration, and password recovery.
- **Workspace Dashboard Layout:** Create a web dashboard with a side navigation menu, top search bar, storage indicator, and workspace layout.
- **File & Folder Management Controls:** Build visual components including:
    - File upload modal with progress state indicators.
    - File inventory list displaying file names, sizes, upload dates, and action menus.
    - Folder navigation and file filtering interfaces.

Milestone 1 Completion Summary

At the conclusion of Milestone 1, you will have a working platform foundation with functional user authentication, an active user database, an interactive web dashboard, and file upload pathways capable of receiving and indexing files

What the document covers

1. **Requirements and architecture** — define user roles, system components and workflows.
2. **Database design** — PostgreSQL, MongoDB and Redis setup.
3. **Project setup** — React/Next.js frontend, FastAPI backend and GitHub repository structure.
4. **Authentication —** registration, login, JWT authentication and role-based access control.
5. **File upload** — validate files, store development copies safely and save metadata.
6. **File management dashboard** — display uploaded files and their details.

7. **Testing** — verify authentication, upload validation and user-level file permissions.
8. **Week-wise plan** — organize the work across two weeks.
9. **Submission checklist** — identify the code, diagrams, documentation and working features required for evaluation.

Milestone 1 Expected Outcomes

1. **Understanding Secure Workflows:** Clear comprehension of secure file-sharing concepts, digital collaboration workflows, and basic system security practices.
2. **Architecture & Database Design Mastery:** Conceptual and practical setup of secure system architecture, along with functional relational (PostgreSQL), document-based (MongoDB), and key-value (Redis) database schemas.
3. **Initialized Project Infrastructure:** Successfully initialized frontend (React/Next.js) and backend (FastAPI) project repositories, connected and running locally (via Docker containerization).
4. **Working Authentication & RBAC System:** A functional, secure user authentication pipeline with password hashing (bcrypt), JSON Web Tokens (JWT), session management, initial MFA capabilities, and Role-Based Access Control (Admin, User, Guest).
5. **Functional File Upload Pipeline:** A backend file intake workflow that validates file sizes and types, accepts files, stores them, and logs file metadata in MongoDB.
6. **Working File Management Interface:** A functional frontend dashboard that lets users log in, view their uploaded files in a table format, create folders, and use file search/filtering UI components.

Deliverables Checklist for Evaluation

During your Milestone 1 review or evaluation, you should be able to demonstrate the following:

- **Architecture Setup:** Demonstrated database services running in Docker and connected to FastAPI.
- **Authentication Flow:** Registering a new account, logging in, receiving a JWT, and accessing restricted routes based on user roles.
- **File Upload Test:** Uploading a sample file through the UI, confirming it passes size/type checks, saves to storage, and creates a metadata entry in MongoDB.
- **UI Workspace:** Navigating through the interactive web dashboard containing file management and folder components.